Modern Cloud Computing

Lecture 21: ML in Cloud II

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Background
- vLLM (SOSP'23)
- Strata (OSDI'26)

Background: How an LLM Generates Text

An autoregressive Transformer factorizes $P(x) = P(x_1)\,P(x_2 \mid x_1) \cdots P(x_n \mid x_1, \ldots, x_{n-1})$ and emits **one token at a time**.

Each layer's self-attention needs the **key and value vectors of every previous token**. Recomputing them each step would be quadratic, so they are cached (the **KV cache**).

Prefill (prompt phase)

- All prompt tokens processed at once
- Matrix–matrix ops → **compute-bound**
- Produces the first token and the prompt's KV cache

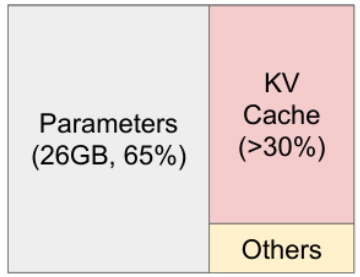
Decode (generation phase)

- One new token per iteration, reads the entire KV cache
- Matrix–vector ops → **memory-bound**
- Dominates request latency

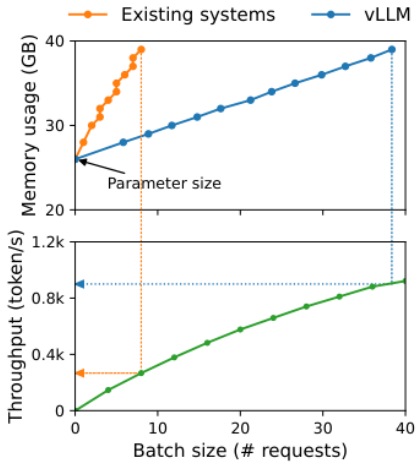
> Serving throughput is bounded by how many requests' KV caches fit in GPU memory at once.

Memory Management Problem

- Example: **65%** of memory is weights, **>30%** is KV cache
- The KV cache **grows and shrinks dynamically** with unknown lifetime and length
- GPU FLOPS grow faster than GPU memory (A100 → H100: >2× FLOPS, same 80 GB)



NVIDIA A100 40GB



(Source: Kwon et al., SOSP '23, Fig. 1)

Part 1

Efficient Memory Management for Large Language Model Serving with PagedAttention

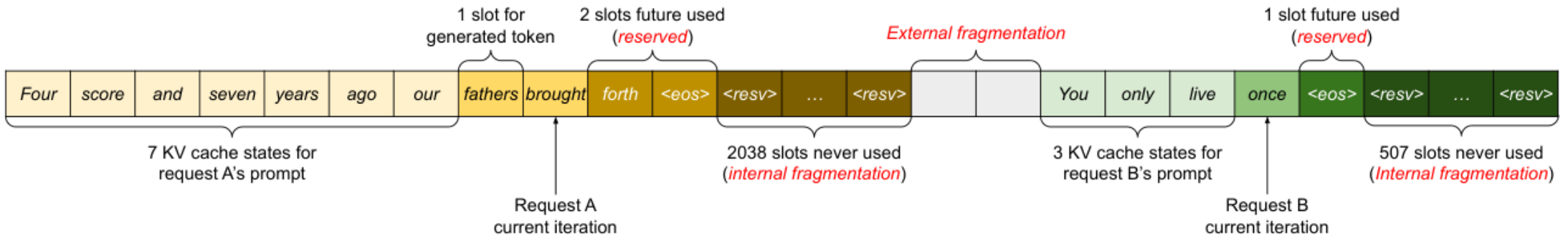
Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica — SOSP 2023

Existing Systems Waste Most of the KV Cache Memory

Deep learning frameworks want tensors **contiguous**

- **Pre-allocate** each request's KV cache at the model's maximum length (e.g., 2,048 tokens):
  - **Reserved** slots for future tokens: unusable by others for the request's whole lifetime
  - **Internal fragmentation**: over-provisioned slots that are never used
  - **External fragmentation**: memory allocator

Only **20.4–38.2%** of KV cache memory holds actual token states; even an *oracle* that knows every output length wastes 62%.



(Source: Kwon et al., SOSP '23, Fig. 2–3)

The Idea: Virtual Memory for the KV Cache

Operating system	vLLM
Page	**KV block** (fixed number of tokens, default 16)
Byte	Token
Process	Request / sequence
Page table	**Block table**: logical block → physical block, # filled
Fork + copy-on-write	Parallel sampling and beam search
Shared library	Shared prompt prefix
Swapping to disk	Swapping KV blocks to CPU RAM, or **recomputation**

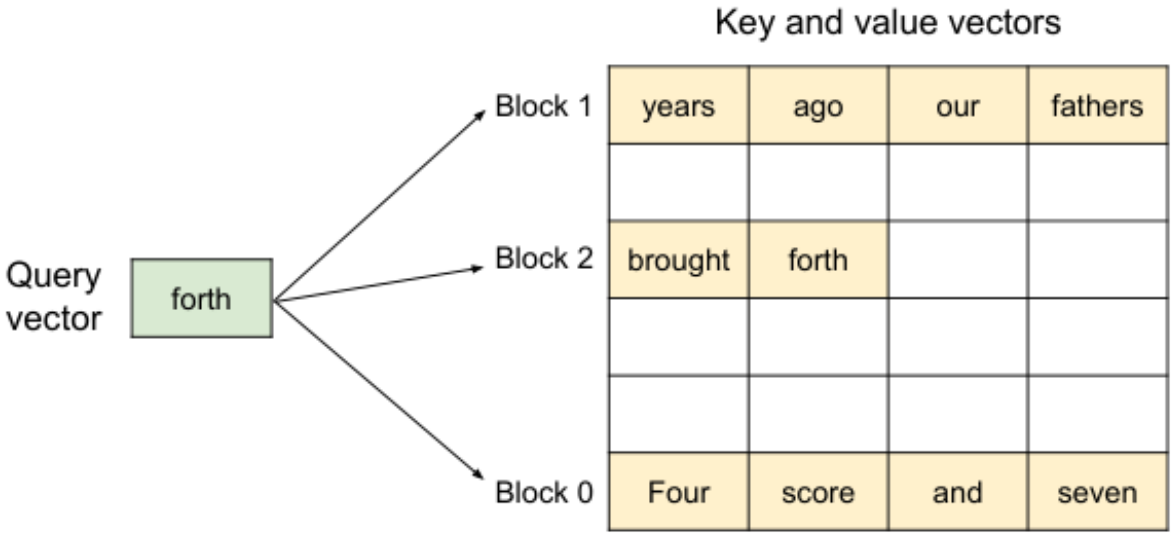
- Blocks are allocated **on demand** as tokens are generated
  - No reservation, waste confined to the last block of each sequence
- All blocks have the same size → **no external fragmentation**
- Physical blocks can be **shared** across sequences via reference counts

PagedAttention

Partition each sequence's KV cache into blocks of B tokens; K_j, V_j are the key/value block j. Attention becomes block-wise:

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \qquad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top$$

- The kernel fetches each block through the block table
  - **Blocks need NOT be contiguous**



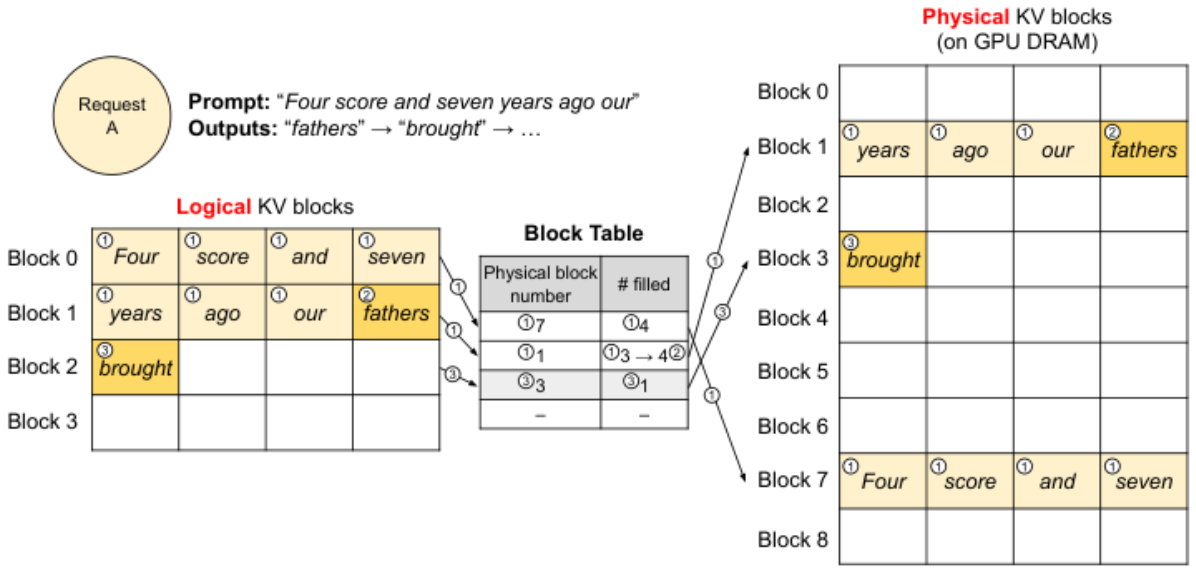
(Source: Kwon et al., SOSP '23, Fig. 5)

Walking Through a Decode Step

Prompt "Four score and seven years ago our" (7 tokens), block size 4:

1. **Prefill**: two logical blocks → physical blocks 7 and 1; block 1 has one free slot
2. **Decode step 1**: "fathers" goes into the free slot of physical block 1; `#filled` updated
3. **Decode step 2**: block full → allocate physical block 3 for logical block 2

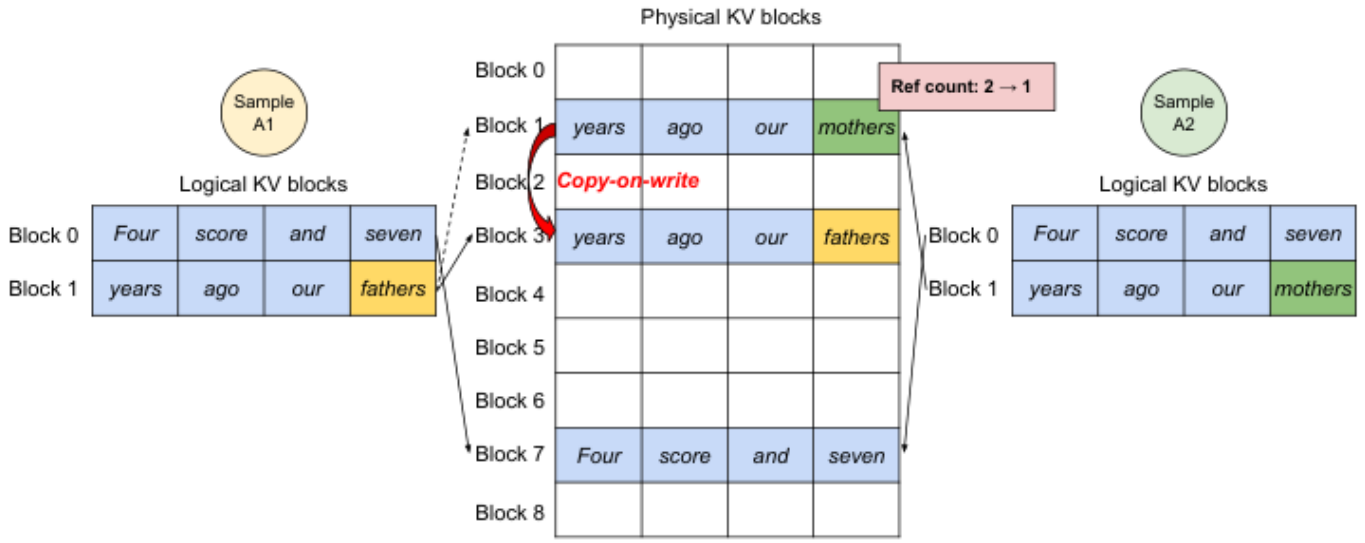
Each iteration, the scheduler sends token IDs **plus block tables** to workers; workers never coordinate memory among themselves.



(Source: Kwon et al., SOSP '23, Fig. 6)

Sharing Across Sequences: Copy-on-Write

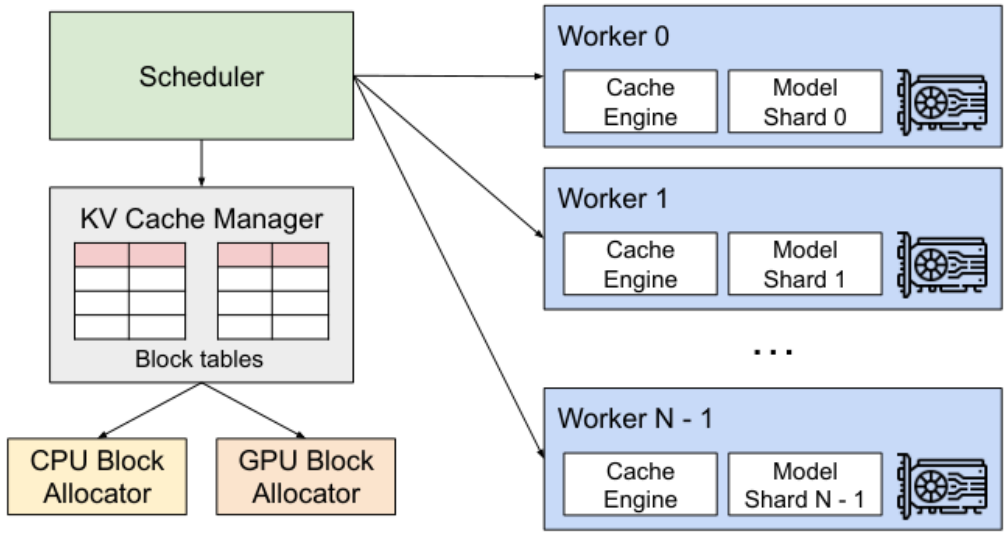
- **Parallel sampling**: samples for one prompt share its physical blocks (ref count 2). When A1 writes to a shared block, vLLM copies it and decrements the count; A2 then writes in place
- **Beam search**: candidates share arbitrary prefixes; blocks freed at ref count 0, so no per-step KV copying
- **Shared system prompt**: reserve physical blocks for a cached prefix; new requests map to them



(Source: Kwon et al., SOSP '23, Fig. 8, 15)

Scheduling, Preemption, Distribution

- **Out of GPU blocks** FCFS: preempt the *latest* arrivals; evict **all or none** of a sequence's blocks (always accessed together)
- **Swapping** evicted blocks to CPU RAM: swap space bounded by the GPU block count
- **Recomputation**: replay prompt + generated tokens as one prefill; ≤20% slower than swapping, better at small block sizes



(Source: Kwon et al., SOSP '23, Fig. 4, 19)

vLLM Takeaways

- LLM serving throughput is **memory-bound**: the batch is limited by how many KV caches fit, not by compute
- Existing systems wasted **60–80%** of KV memory to reservation and fragmentation
- **PagedAttention** decouples logical from physical KV storage → near-zero waste, flexible sharing via copy-on-write
- Classic OS ideas (paging, page tables, CoW, swapping) transfer with LLM-specific twists: all-or-nothing eviction, recomputation instead of swap-in
- vLLM became the de-facto open-source serving engine; "paged KV cache" is now standard in TensorRT-LLM, SGLang, and others

> But paging was designed to solve *fragmentation inside one GPU*. What happens when the KV cache no longer fits in the GPU at all?

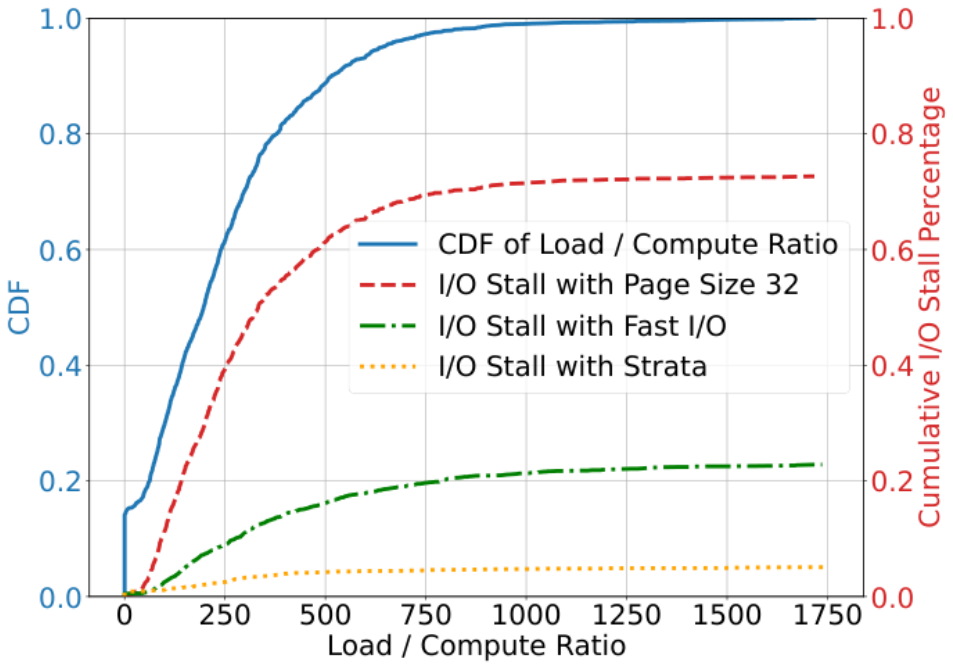
Part 2

Strata: Hierarchical Context Caching for Long Context Language Model Serving

Xie, Xu, Zhao, An, Mailthody, Mahlke, Garland, Kozyrakis — OSDI 2026

Long Contexts and Context Caching

- Frontier models accept **128K–1M+** tokens: coding agents, RAG, document analysis, multi-turn agents
- Recomputing attention over a million-token context is prohibitive → **context (prefix) caching**: keep the KV cache of a reused prefix and skip its prefill (OpenAI, Google, Anthropic APIs all offer it)
- But **40 GB of HBM holds only ~0.3M tokens** of Llama-8B KV cache → production systems build a **hierarchy**: GPU HBM → CPU DRAM → SSD → remote pools
- Every cache hit now means **loading** KV pages back into the GPU before prefill can run

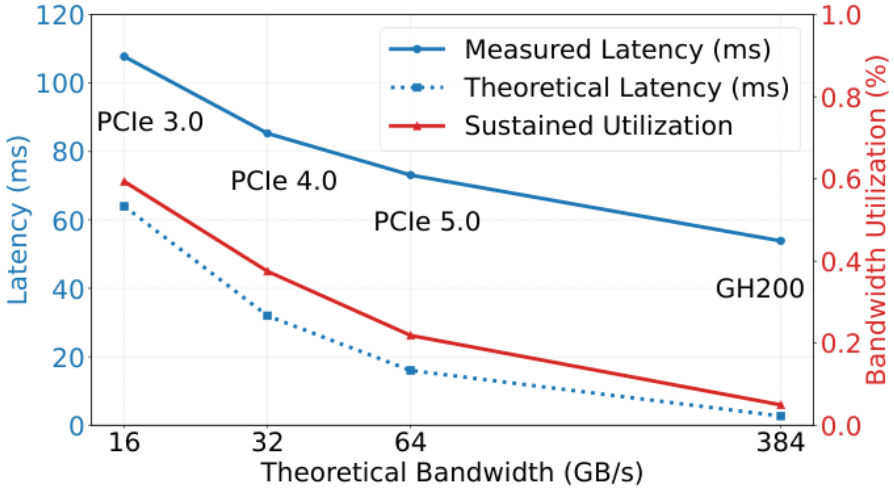


(Source: Xie et al., OSDI '26, Fig. 1)

Problem 1: Paging Fragments the KV Cache for I/O

Little's law: I/O throughput $X = C \cdot S/L$.

- C: Average number of concurrent I/O operations, S: Average data size/IO operation, L: Average latency per operation
  - With `cudaMemcpyAsync`, the practical knob is **transfer size** S
  - LLM engines use **tiny pages** (1–32 tokens) for hit rate; one page's K/V spans **a few KB**
    - Bigger pages hurt caching



Problem 2: Schedulers Are Blind to Loading Cost

Prior systems overlap **layer $N+1$'s** KV load with **layer N's** prefill, assuming compute hides I/O

- With long cached contexts and few new tokens, loading dominates: even with fast I/O, **24%** of prefill time stalls on cache loading
- The **load/compute ratio** (tokens loaded ÷ new tokens) is heavy-tailed

Delay hits: many requests for the same context arrive while its cache miss is still being resolved, and each is treated as a miss and recomputes the prefix

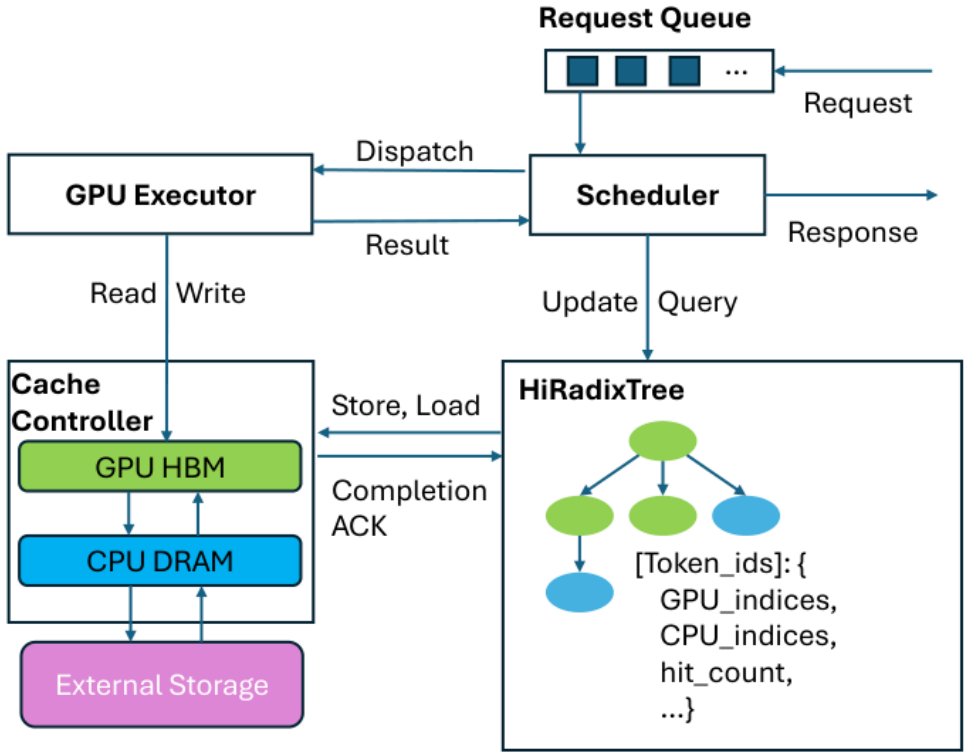
- Asynchronous schedulers that prepare the next batch early widen the window

(Source: Xie et al., OSDI '26, §3, Fig. 1)

Strata Architecture

Built into **SGLang**, deployed in production.

- **Cache Controller** (data plane): GPU-assisted I/O between HBM and CPU DRAM; per-tier layouts; prefetch from external storage; configurable write-back policies
- **Scheduler** (control plane): cache-aware batch formation: defer delay hits, balance load vs. compute, fill stalls
- **HiRadixTree**: SGLang's radix tree extended into a page table across tiers: per node: GPU indices, CPU indices, hit count, *transient* markers



(Source: Xie et al., OSDI '26, Fig. 4)

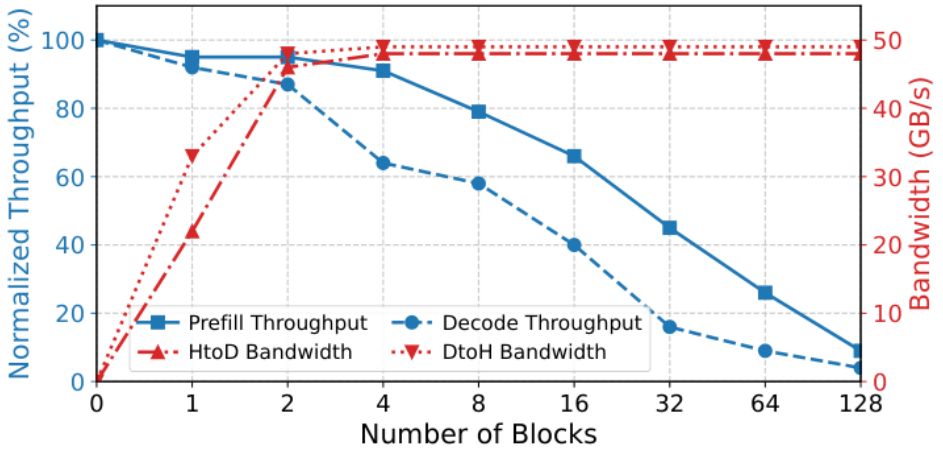
Technique 1: GPU-Assisted I/O

Instead of many small `cudaMemcpyAsync` calls, launch a **CUDA kernel**

- Launch thousands of threads
- Copy a small chunk of data to its local registers and stream to destination

Advantanges:

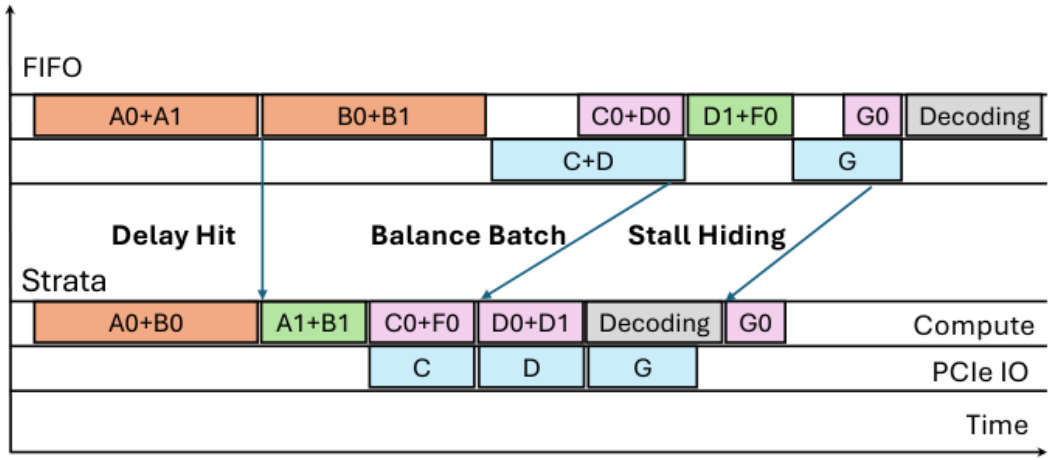
- Enhanced concurrency
- Compatible with small transfer size
- Enables flexible memory layout



(Source: Xie et al., OSDI '26, Fig. 5)

Technique 2: Cache-Aware Scheduling

- **Defer delay hits**
- **Balanced batches**: Forming batches to minimize loading waits
- **Bubble filling**: Overlapping decoding with data movement



(Source: Xie et al., OSDI '26, Fig. 7, Alg. 1)